

Costmate V3

AI-Powered Civil Estimation Platform

Agentic Architecture & System Overview

1. Overview

Costmate is a web application that automates civil engineering and construction estimation. Using an Agentic AI Swarm, it analyzes architectural floor plans (PDFs or images), extracts structural dimensions and elements, and automatically generates a complete Bill of Quantities (BOQ) and cost estimate.

The system is orchestrated end-to-end by LangGraph, processing blueprints, specifications, and schedules through a layered pipeline of specialized AI agents.

2. Key Features

- **Automated Blueprint Analysis** — a multi-agent AI pipeline (powered by OpenRouter VLMs) scans PDF schedules and floor plans to extract room dimensions, beams, columns, doors, windows, and footings.
- **Intelligent Schedule Parsing** — dynamically parses complex, multi-page schedules (e.g. Door & Window schedules) using a unified, dynamic Pydantic extraction schema.
- **Human-in-the-Loop QA** — a glassmorphic UI presents extracted data for user review before final calculations; a dedicated Review Queue Modal flags unresolved items for human verification.
- **AI Swarm Copilot** — an integrated AI chat sidebar lets users instruct the copilot to modify dimensions or add missing parameters directly.
- **Dynamic Excel Export** — finalized data is compiled into a professional, multi-sheet Excel BOQ, previewable in-browser and downloadable as .xlsx.
- **Plan Annotation** — automatically generates a downloadable version of the floor plan with detected doors and windows color-coded and highlighted.

3. Tech Stack

Frontend

- Framework: Next.js (React)
- Styling: Tailwind CSS — dark-mode aesthetics with glassmorphism
- Key Components: interactive TabEditor (schedules), ReviewQueueModal (human QA), real-time CopilotPanel

Backend

- Framework: Python / FastAPI
- Database: PostgreSQL — sessions, drafts, and LangGraph checkpoints
- AI Orchestration: LangGraph + OpenRouter API (large reasoning / vision-language models)
- Processing: PyMuPDF for document ingestion, Pandas for Excel generation

4. Layer-Wise Architecture

Costmate is built on a structured, layered multi-agent swarm orchestrated by LangGraph. Plans, specifications, and schedules move through five sequential layers, with a human verification gate before final output generation.

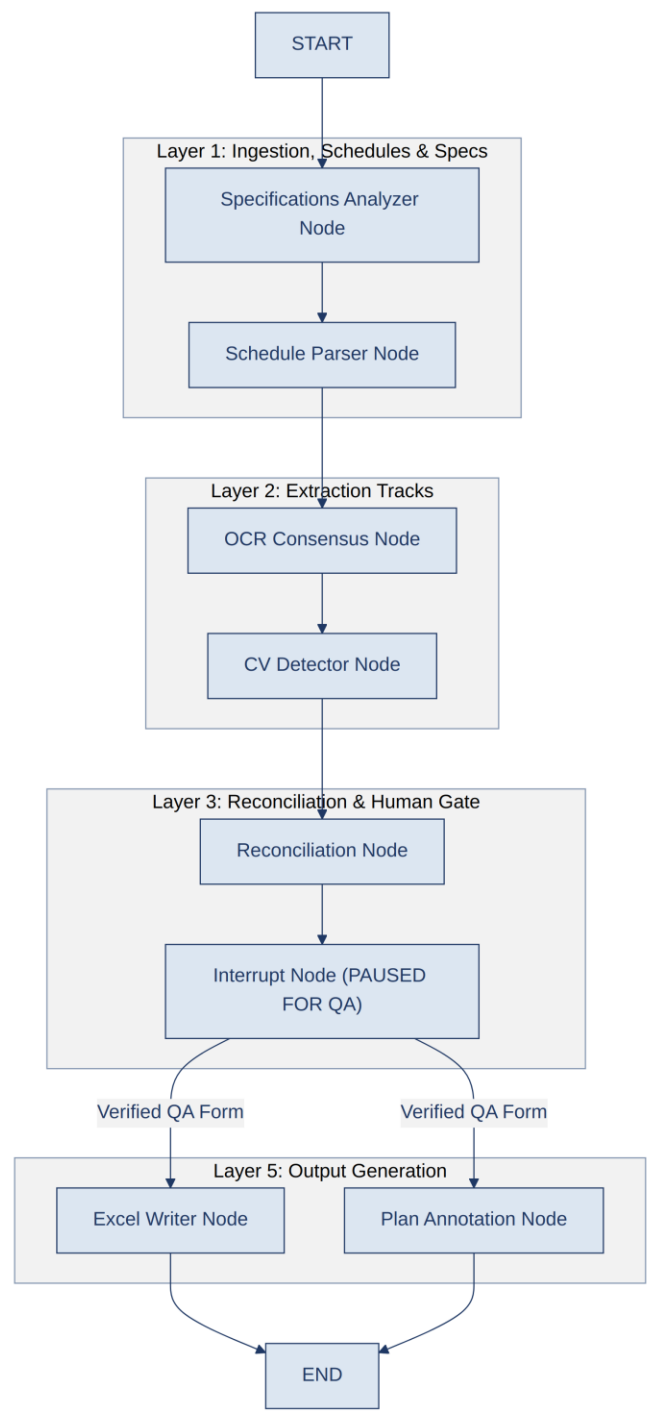


Figure 1 — Layer-wise execution graph, from ingestion through the human QA gate to output generation.

5. Layer & Agent Responsibilities

Layer	Agent / Node	Input	Output	Role	Why We Use It
1	Specifications Analyzer	Raw .docx / .txt spec files	specifications_insights (JSON)	Extracts hardware exclusions, materials, and project constraints.	Filters out materials outside the estimator's scope to prevent wrong pricing.
1	Schedule Parser	Uploaded schedule sheets (Image/PDF)	schedule_data (JSON)	Extracts door/window schedule tables (sizes, counts, frames) via dual-pass Qwen-72B VLM.	Digitizes structural schedule parameters into a canonical format.
2	OCR Consensus	Uploaded plan drawings (PDF/Image)	ocr_results (Text list)	Multi-pass text parser extracting text labels and room locations from plans.	Ensures room tags and numbers are extracted with high character accuracy.
2	CV Detector	Plan drawings + schedule_data	cv_results (JSON list)	Matches marks (e.g. D1, W2) to layouts, cropping images and classifying swing directions.	Physically locates door/window icons on drawings and detects opening swing modes.
3	Reconciliation Node	Outputs from Layers 1 & 2	qa_prefilled (JSON) + unresolved_queue	Merges OCR text and CV detection results; flags mismatching doors for human check.	Combines visual/textual data and structures the review dashboard.
3	Interrupt Node	Prefilled QA schema	User input updates via UI	Halts the LangGraph run until the user validates the parameters.	Acts as the Human Gate to ensure 100% precision before final calculations.
5	Excel Writer	Verified Q&A Form	Formatted .xlsx BOQ	Generates Estimation and Raw schedules with correct column ordering.	Builds the final customer-ready Microsoft Excel takeoff sheet.
5	Plan Annotation	Verified Q&A Form + Plans	Annotated plan drawings	Overlays green/red pins on plans pointing to located door/window openings.	Provides visual validation of all detected items on the layout drawings.

6. End-to-End Execution Sequence

The diagram below traces a single project run from upload through the human QA gate to final BOQ delivery.

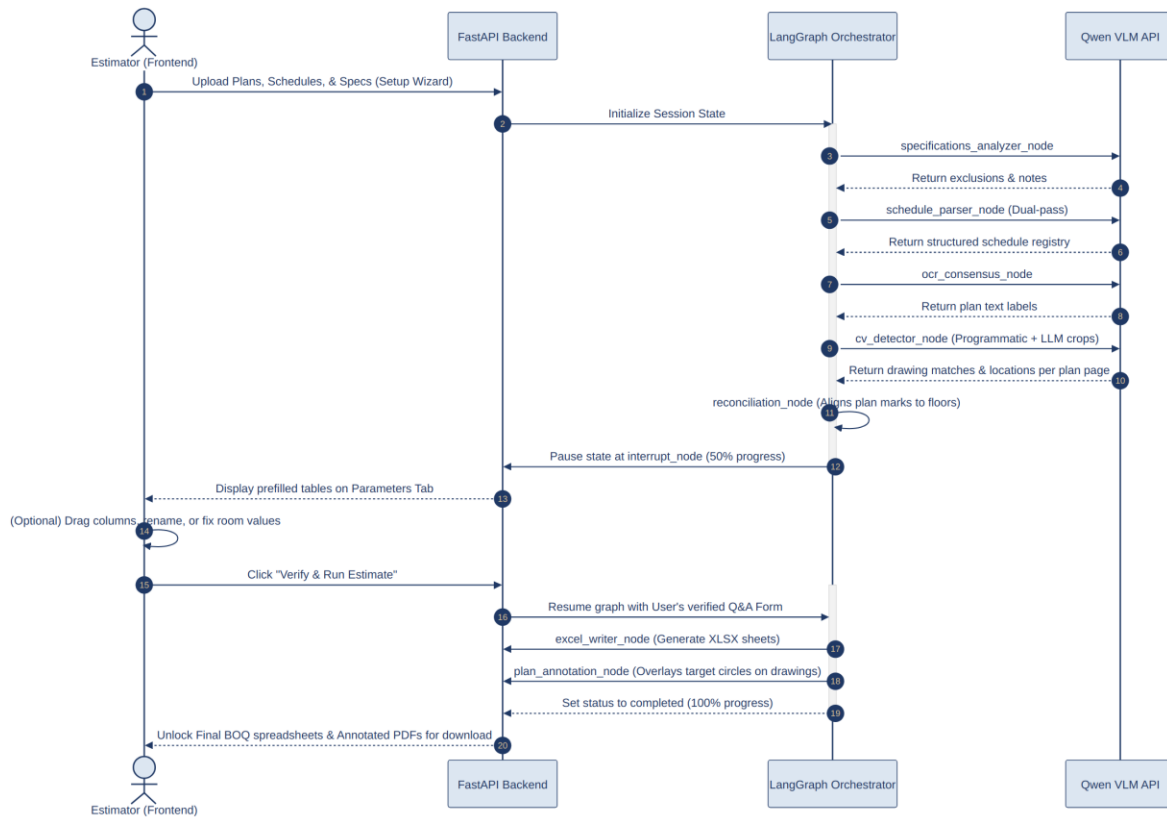


Figure 2 — Sequence of interactions between the Estimator, FastAPI Backend, LangGraph Orchestrator, and Qwen VLM API.

Flow of Execution — Step by Step

1. Estimator uploads Plans, Schedules, and Specs via the Setup Wizard (Frontend).
2. FastAPI Backend initializes the session state.
3. LangGraph Orchestrator calls `specifications_analyzer_node`; Qwen VLM API returns exclusions and notes.
4. Orchestrator calls `schedule_parser_node` (dual-pass); VLM returns the structured schedule registry.
5. Orchestrator calls `ocr_consensus_node`; VLM returns plan text labels.
6. Orchestrator calls `cv_detector_node` (programmatic + LLM crops); VLM returns drawing matches and locations per plan page.
7. Orchestrator runs `reconciliation_node`, aligning plan marks to floors.
8. Orchestrator pauses execution at `interrupt_node` (50% progress).
9. Backend displays the prefilled tables on the Parameters Tab to the Estimator.
10. Estimator optionally drags columns, renames fields, or fixes room values.
11. Estimator clicks "Verify & Run Estimate."
12. Backend resumes the graph with the user's verified Q&A Form.
13. Orchestrator runs `excel_writer_node`, generating XLSX sheets.
14. Orchestrator runs `plan_annotation_node`, overlaying target circles on drawings.
15. Orchestrator sets status to completed (100% progress).
16. Backend unlocks the final BOQ spreadsheets and annotated PDFs for download.

